

GENERATIVE AI ASSESSMENT

Question 13 – Extractive Question Answering

Register Number	192472137
Topic	Extractive Question Answering
AI Platform	Hugging Face
Model	deepset/roberta-base-squad2

1. Assessment Question

13. Extractive Question Answering

An HR team answers policy questions from an employee handbook page.

Task: Using a Hugging Face question-answering model, answer questions against a provided passage. Demonstrate three answerable questions and one whose answer is not in the passage.

2. Objective

To develop an Extractive Question Answering system using the Hugging Face Inference API and the `deepset/roberta-base-squad2` model. The system accepts an HR employee handbook passage and a question, extracts the most relevant answer directly from the passage, provides a confidence score, and identifies questions whose answers are not present in the supplied passage.

3. Tools and Technologies

Component	Details
Programming Language	Python
AI Platform	Hugging Face
API	Hugging Face Inference API
Model	<code>deepset/roberta-base-squad2</code>
Python Library	<code>huggingface_hub</code>
Task	Extractive Question Answering
Development Environment	Visual Studio Code
Authentication	<code>HF_TOKEN</code> environment variable

4. Model and Parameter Details

The model selected is `deepset/roberta-base-squad2`. It is an extractive question-answering model based on RoBERTa and fine-tuned for SQuAD 2.0. SQuAD 2.0 includes both answerable and unanswerable questions, making the model suitable for this assessment.

Parameter	Value
Model	<code>deepset/roberta-base-squad2</code>
Provider	<code>hf-inference</code>
Task	Question Answering
<code>top_k</code>	1 / default single best answer
Confidence threshold	0.20
Authentication	<code>HF_TOKEN</code> environment variable

The model is appropriate because the task requires answers to be extracted from the supplied passage rather than generated using outside information. The confidence threshold is used to reject very low-confidence predictions, which is useful for the unanswerable case.

5. Provided HR Employee Handbook Passage

The company provides all full-time employees with 20 days of paid annual leave every calendar year. Employees may request annual leave through the company's HR portal at least five working days before the planned leave date. Emergency leave may be requested with shorter notice and must be approved by the employee's manager.

Employees who work for more than six months are eligible for the company's health insurance plan. The company provides health insurance coverage for the employee and allows employees to add eligible family members by paying the applicable additional premium.

The standard working hours are from 9:00 AM to 6:00 PM, Monday through Friday, with a one-hour lunch break.

Employees must inform their manager as soon as possible if they are unable to attend work because of illness. For extended medical leave, employees may be required to submit appropriate medical documentation to the HR department.

The company conducts an annual performance review for all full-time employees. Performance reviews are normally completed during the month of December.

Employees are expected to maintain confidentiality regarding company information, customer information, and internal business documents.

6. Methodology / Request–Response Flow

1. Load the Hugging Face API token from the HF_TOKEN environment variable.
2. Create an InferenceClient using the Hugging Face Inference API.
3. Store the HR employee handbook passage as the context.
4. Prepare each question and send the question together with the passage to the question-answering model.
5. Receive the model response containing the extracted answer and confidence score.
6. Compare the confidence score with the threshold of 0.20.
7. Display the extracted answer when confidence is sufficient; otherwise report that the answer was not found in the passage.
8. Handle missing input, authentication/API failures, rate-limit or quota errors, and timeout errors.

7. Test Cases

No.	Question	Type	Expected Information
1	How many days of paid annual leave do full-time employees receive every year?	Answerable	20 days
2	How many working days before planned leave should an employee submit a request?	Answerable	five working days
3	What are the standard working hours?	Answerable	9:00 AM to 6:00 PM
4	What is the maximum	Unanswerable	Not present in passage

	salary increase an employee can receive?		
--	--	--	--

8. Actual Program Output

The following output was obtained from the working program execution:

Model: deepset/roberta-base-squad2
Task: Extractive Question Answering
Provider: Hugging Face Inference API

Question: How many days of paid annual leave do full-time employees receive every year?
Answer: 20
Confidence: 0.7845
Status: Answer found in passage

Question: How many working days before planned leave should an employee submit a request?
Answer: five
Confidence: 0.5264
Status: Answer found in passage

Question: What are the standard working hours?
Answer: 9:00 AM to 6:00 PM
Confidence: 0.4551
Status: Answer found in passage

Question: What is the maximum salary increase an employee can receive?
Answer: Answer not found in the provided passage.
Confidence: 0.0002
Status: Answer not present / low confidence

The exact extracted span may contain only the most relevant words. For example, the model returned "20" for the first question and "five" for the second question. These are valid extractive answers because they are taken directly from the supplied passage.

9. Error Handling

Error	Handling
Missing API key	The program checks whether HF_TOKEN exists before creating the Hugging Face client.
Invalid / empty input	The program checks whether the question is empty before sending the request.
Authentication failure	HTTP 401/403-type errors are detected and reported.
Rate-limit / quota error	Rate or quota-related API errors are detected and reported.
Timeout	Timeout-related failures are detected and reported.
Other API failure	Unexpected API exceptions are caught and displayed instead of terminating without explanation.

10. API Key Security

The Hugging Face access token is not hard-coded in the submitted Python program. The program retrieves it using `os.getenv("HF_TOKEN")`. The token is supplied through an environment variable in PowerShell. This prevents the secret key from being exposed directly in the source code.

```
token = os.getenv("HF_TOKEN")
```

11. Complete Python Program

```
import os
from huggingface_hub import InferenceClient

MODEL_NAME = "deepset/roberta-base-squad2"
CONFIDENCE_THRESHOLD = 0.20

PASSAGE = """
The company provides all full-time employees with 20 days of
paid annual leave every calendar year. Employees may request
annual leave through the company's HR portal at least five
working days before the planned leave date. Emergency leave
may be requested with shorter notice and must be approved by
the employee's manager.

Employees who work for more than six months are eligible for
the company's health insurance plan. The company provides
health insurance coverage for the employee and allows
employees to add eligible family members by paying the
applicable additional premium.

The standard working hours are from 9:00 AM to 6:00 PM,
Monday through Friday, with a one-hour lunch break.

Employees must inform their manager as soon as possible if
they are unable to attend work because of illness. For
extended medical leave, employees may be required to submit
appropriate medical documentation to the HR department.

The company conducts an annual performance review for all
full-time employees. Performance reviews are normally
completed during the month of December.

Employees are expected to maintain confidentiality regarding
company information, customer information, and internal
business documents.
"""

def create_client():
    token = os.getenv("HF_TOKEN")

    if not token:
        raise ValueError(
            "HF_TOKEN is missing. Please set your Hugging Face "
            "API token as an environment variable."
        )

    return InferenceClient(
        provider="hf-inference",
```

```

        api_key=token
    )

def answer_question(client, question):

    if not question.strip():
        raise ValueError("Question cannot be empty.")

    try:
        result = client.question_answering(
            question=question,
            context=PASSAGE,
            model=MODEL_NAME
        )

        answer = result.answer
        score = result.score

        if not answer or score < CONFIDENCE_THRESHOLD:
            return {
                "answer": "Answer not found in the provided passage.",
                "score": score,
                "found": False
            }

        return {
            "answer": answer,
            "score": score,
            "found": True
        }

    except Exception as error:

        error_message = str(error).lower()

        if "401" in error_message or "403" in error_message:
            raise RuntimeError(
                "Authentication error. Please check your HF_TOKEN."
            )

        elif "rate" in error_message or "quota" in error_message:
            raise RuntimeError(
                "API rate limit or quota exceeded."
            )

        elif "timeout" in error_message:
            raise RuntimeError(
                "The request timed out. Please try again."
            )

        else:
            raise RuntimeError(
                f"Hugging Face API error: {error}"
            )

def display_result(number, question, result):

    print("\n" + "=" * 70)
    print(f"QUESTION {number}")
    print("=" * 70)

```

```

print("Question:", question)

if result["found"]:
    print("Answer:", result["answer"])
    print("Confidence:", round(result["score"], 4))
    print("Status: Answer found in passage")
else:
    print("Answer:", result["answer"])
    print("Confidence:", round(result["score"], 4))
    print("Status: Answer not present / low confidence")

def main():

    print("=" * 70)
    print("HR EMPLOYEE HANDBOOK")
    print("EXTRACTIVE QUESTION ANSWERING SYSTEM")
    print("=" * 70)

    print("\nModel:", MODEL_NAME)
    print("Task: Extractive Question Answering")
    print("Provider: Hugging Face Inference API")

    try:
        client = create_client()

    except ValueError as error:
        print("\nERROR:", error)
        return

    questions = [
        "How many days of paid annual leave do full-time employees receive every year?",
        "How many working days before planned leave should an employee submit a request?",
        "What are the standard working hours?",
        "What is the maximum salary increase an employee can receive?"
    ]

    for number, question in enumerate(questions, start=1):

        try:
            result = answer_question(client, question)

            display_result(
                number,
                question,
                result
            )

        except (ValueError, RuntimeError) as error:

            print("\n" + "=" * 70)
            print(f"QUESTION {number}")
            print("=" * 70)

            print("Question:", question)
            print("ERROR:", error)

    print("\n" + "=" * 70)
    print("ASSESSMENT COMPLETED")
    print("=" * 70)

```

```
if __name__ == "__main__":  
    main()
```

12. Result

The Extractive Question Answering system successfully answered three questions using information contained in the HR employee handbook passage. The fourth question asked for information about the maximum salary increase, which was not available in the passage. The model produced a very low confidence score of 0.0002 for this question, and the program correctly reported that the answer was not found.

13. Conclusion

The assessment successfully demonstrates an Extractive Question Answering application using Hugging Face. The system follows the complete request-response flow, securely loads the API key, extracts answers from the supplied HR passage, provides confidence scores, identifies an unanswerable question, and includes handling for common API and input errors. The selected RoBERTa SQuAD2 model is suitable because the task requires extracting information from a given passage rather than generating unrelated information.

14. Short Viva Explanation

I developed an Extractive Question Answering system for an HR employee handbook using the Hugging Face Inference API and the deepset/roberta-base-squad2 model. The program takes a passage and a question, sends them to the model, and extracts the most relevant answer directly from the passage. I demonstrated three answerable questions and one unanswerable question. Confidence scores are used to identify low-confidence answers. The Hugging Face API token is stored in the HF_TOKEN environment variable rather than being hard-coded. The program also handles missing input, authentication failures, rate limits, timeouts, and other API errors.